

Física Computacional

ano letivo: 2021/2022

docente: Nuno Castro [nuno.castro@fisica.uminho.pt]

Licenciatura em Física e Licenciatura / Mestrado Integrado em Engenharia Física

11ª aula (6/1/2022)



Integração de Verlet

$$F = ma = m \frac{d^2 x}{dx^2}$$

$$\begin{aligned} \frac{\Delta^2 \vec{x}_n}{\Delta t^2} &= \frac{\frac{\vec{x}_{n+1} - \vec{x}_n}{\Delta t} - \frac{\vec{x}_n - \vec{x}_{n-1}}{\Delta t}}{\Delta t} \\ &= \frac{\vec{x}_{n+1} - 2\vec{x}_n + \vec{x}_{n-1}}{\Delta t^2} \\ &= \vec{a}_n = A(\vec{x}_n) \end{aligned}$$

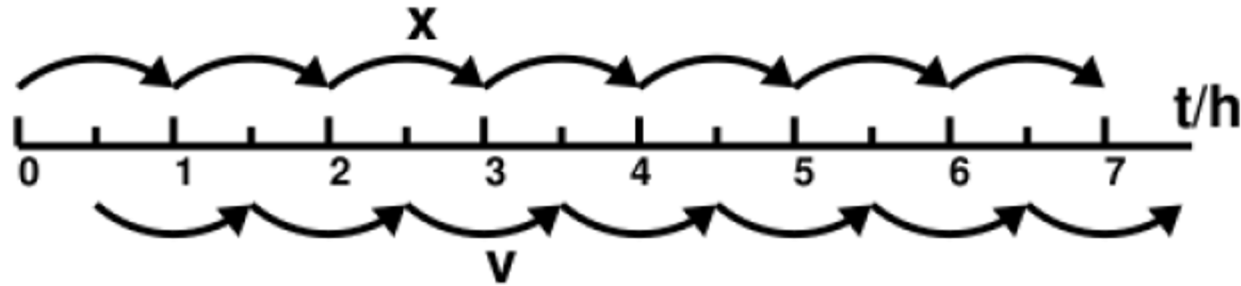
$$\vec{x}_{n+1} = 2\vec{x}_n - \vec{x}_{n-1} + \vec{a}_n \Delta t^2, \quad \vec{a}_n = A(\vec{x}_n)$$

$$\vec{x}(t + \Delta t) = \vec{x}(t) + \vec{v}(t)\Delta t + \frac{\vec{a}(t)\Delta t^2}{2} + \frac{\vec{b}(t)\Delta t^3}{6} + \mathcal{O}(\Delta t^4)$$

$$\vec{x}(t - \Delta t) = \vec{x}(t) - \vec{v}(t)\Delta t + \frac{\vec{a}(t)\Delta t^2}{2} - \frac{\vec{b}(t)\Delta t^3}{6} + \mathcal{O}(\Delta t^4)$$

Integração de Verlet e *leapfrog*

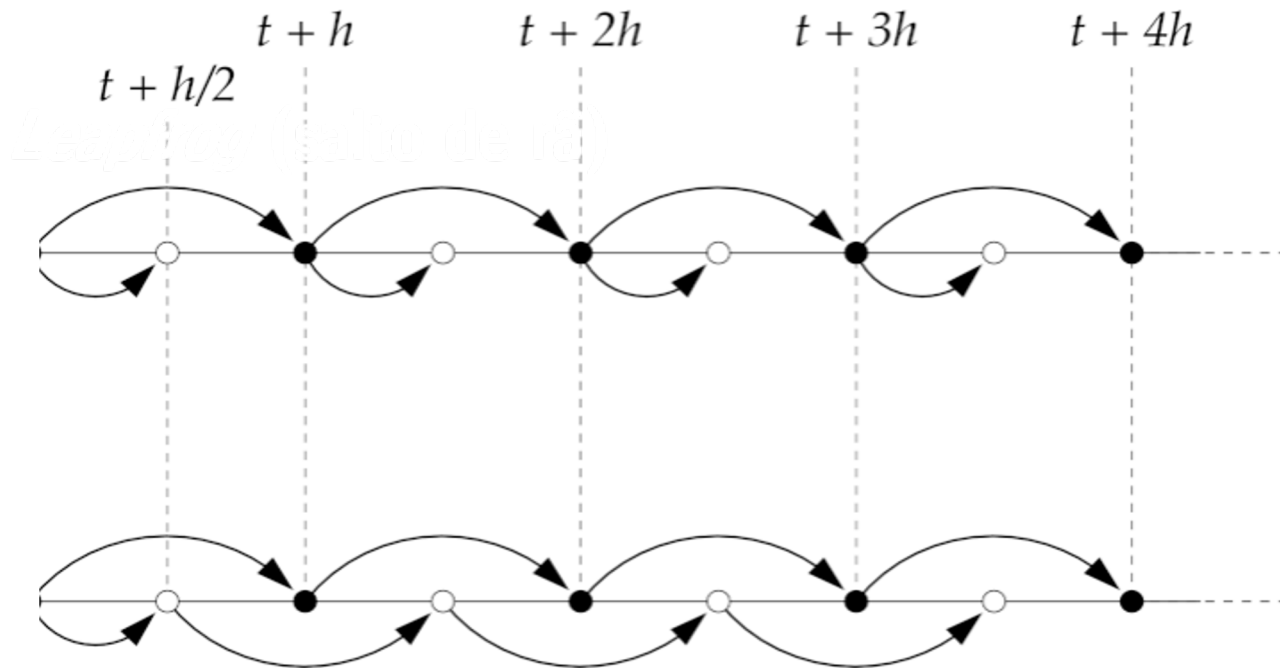
$$\vec{x}(t + \Delta t) = 2\vec{x}(t) - \vec{x}(t - \Delta t) + \vec{a}(t)\Delta t^2 + \mathcal{O}(\Delta t^4)$$



Leap-frog (salto de rã)

Runge-kutta de
2ª ordem

leapfrog



Integração de Verlet - velocidade

$$\frac{dx}{dt} = v$$

$$\frac{dv}{dt} = F(x) = \left(-\frac{dU}{dx}\right)$$

$$x_1 = x_0 + v_{1/2}h$$

$$v_{3/2} = v_{1/2} + F(x_1)h$$

Integração de Verlet - velocidade

$$x_{n+1} = x_n + v_{n+1/2}h$$

$$v_{n+1/2} = v_{n-1/2} + a(x_n)h$$

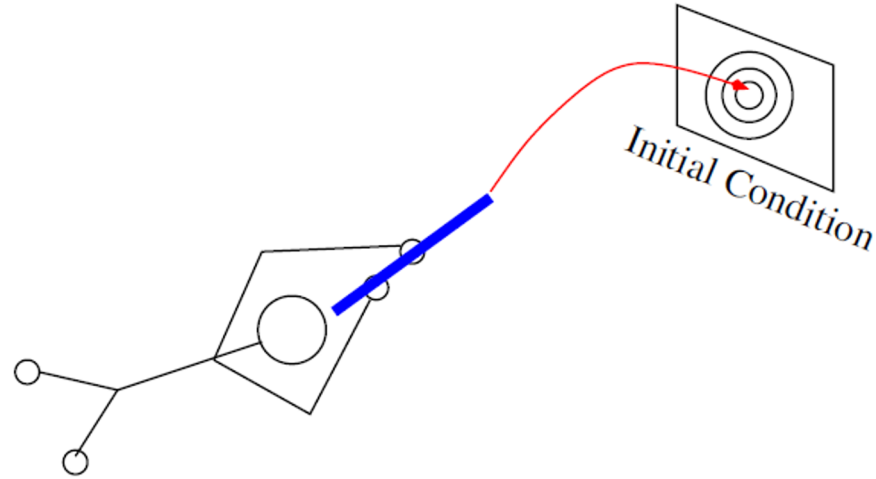
$$v_{n+1/2} = v_n + \frac{1}{2}ha(x_n)$$

Alternativamente:

$$x_{n+1} = x_n + hv_{n+1/2}$$

$$v_{n+1} = v_{n+1/2} + \frac{1}{2}ha(x_{n+1})$$

Método dos disparos



Ideia básica: pesquisa por *tentativa/erro* dos valores das condições iniciais que correspondem às condições fronteiras dadas

Método dos disparos

$$\frac{d^2 y_1}{dx^2} = f(x, y_1, y_1')$$



$$y_1' \equiv \frac{dy_1}{dx} = y_2$$

$$\frac{dy_2}{dx} = f(x, y_1, y_2)$$

$$y_1(x_i) = u_0$$

$$y_1(x_f) = u_1 \quad \left| \begin{array}{l} \text{condição} \\ \text{fronteira} \end{array} \right.$$

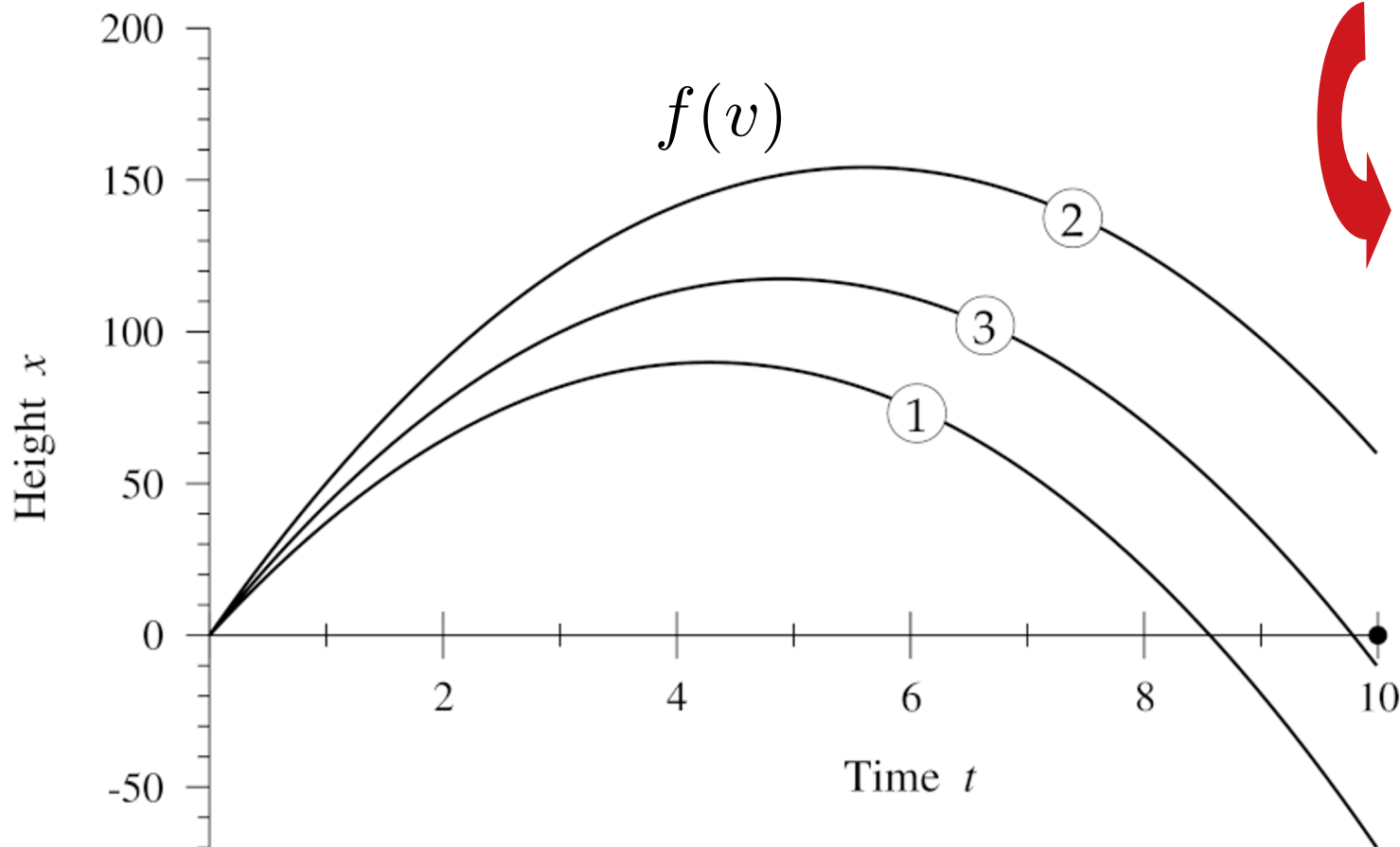
Método dos disparos

- dado $y_1(x_i)$
- adivinhar $y_1'(x_i) = y_2(x_i) = \alpha$
- o parâmetro α é ajustado para satisfazer $y_1(x_f) = u_1$
- para tal pode-se usar e.g. o método da secante ou da bissecção

$$g(\alpha) \equiv u_\alpha(x_f) - u_1$$

$$g(\alpha) = 0$$

Método dos disparos



Lançamento de projétil na vertical:

- sabemos a posição inicial (mas não a velocidade)
- sabemos a posição final

$$\frac{d^2x}{dt^2} = -g$$

$$\begin{cases} \frac{dx}{dt} = y \\ \frac{dy}{dt} = -g \end{cases}$$

Método dos disparos

```
1 from numpy import array,arange
2
3 g = 9.81      # Acceleration due to gravity
4 a = 0.0       # Initial time
5 b = 10.0      # Final time
6 N = 1000     # Number of Runge-Kutta steps
7 h = (b-a)/N   # Size of Runge-Kutta steps
8 target = 1e-10 # Target accuracy for binary search
9
10 # Function for Runge-Kutta calculation
11 def f(r):
12     x = r[0]
13     y = r[1]
14     fx = y
15     fy = -g
16     return array([fx,fy],float)
17
18 # Function to solve the equation and calculate the final height
19 def height(v):
20     r = array([0.0,v],float)
21     for t in arange(a,b,h):
22         k1 = h*f(r)
23         k2 = h*f(r+0.5*k1)
24         k3 = h*f(r+0.5*k2)
25         k4 = h*f(r+k3)
26         r += (k1+2*k2+2*k3+k4)/6
27     return r[0]
```

```
28
29 # Main program performs a binary search
30 v1 = 0.01
31 v2 = 1000.0
32 h1 = height(v1)
33 h2 = height(v2)
34
35 while abs(h2-h1)>target:
36     vp = (v1+v2)/2
37     hp = height(vp)
38     if h1*hp>0:
39         v1 = vp
40         h1 = hp
41     else:
42         v2 = vp
43         h2 = hp
44
45 v = (v1+v2)/2
46 print("The required initial velocity is",v,"m/s")
```

The required initial velocity is 49.049999999999815 m/s

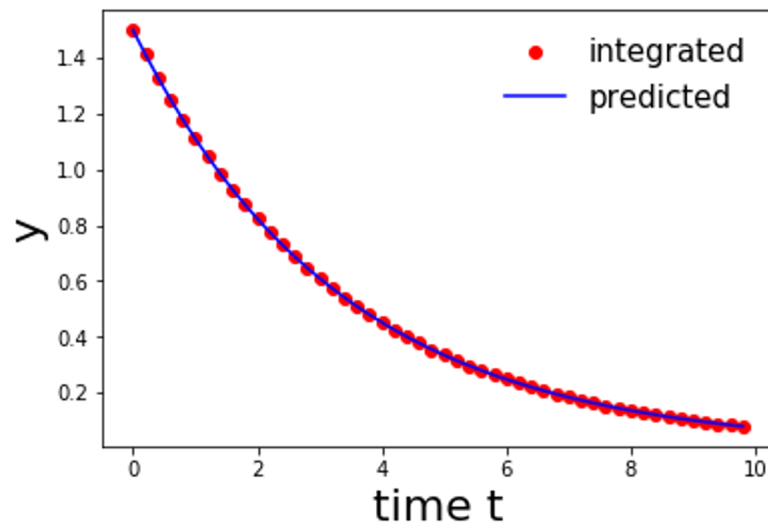
odeint (scipy)

```
1 %matplotlib inline
2 import numpy as np
3 from scipy.integrate import odeint
4 import matplotlib.pyplot as plt
5
6 y0 = 1.5
7 a=0.3
8
9 tmax=10.0;
10 dt = 0.2;
11 tt = np.arange(0,tmax,dt);
12
13 def func(y,t):
14     return -a*y
15
16 # solve the ordinary differential equation
17 yy=odeint(func,y0,tt) # func is our function, y0 is initial condition,
18 # tt a time vector
19 # yy is now a vector yy values computed by integrating the equation
20 # we will have a y value at each time in the vector yy
21
22 ysol = y0*np.exp(-a*tt)
23 plt.plot(tt,yy,'ro',label="integrated")
24 plt.plot(tt,ysol,'b-',label="predicted")
25 plt.xlabel("time t",fontsize=22)
26 plt.ylabel("y",fontsize=22)
27 plt.legend(loc='best',frameon=False,fontsize=15)
28
29 # how well did the integration do?
30 print ('error ', yy[-1] - ysol[-1])
```

error [-1.14280241e-08]

$$\frac{dy}{dt} = -ay$$

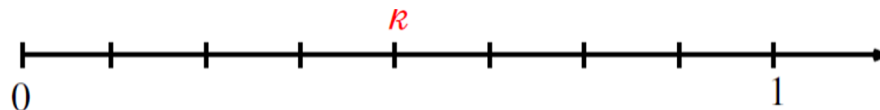
$$y(t) = y_0 e^{-at}$$



Método da relaxação

$$\frac{d^2 y}{dx^2} = f(x, y) \Rightarrow \frac{d^2 y}{dx^2} - f(x, y) = 0$$

1) dividir o intervalo em N



2) usar a definição (numérica) de derivada de 2ª ordem

respeitando as
condições fronteira
pretendidas

$$x = x_k \quad \Rightarrow \quad \frac{y_{k+1} - 2y_k + y_{k-1}}{h^2} - f(x_k, y_k) = 0$$

$$y_{k+1} - 2y_k + y_{k-1} - h^2 f(x_k, y_k) = 0$$

$$y_k = \frac{y_{k+1} + y_{k-1} - h^2 f(x_k, y_k)}{2}$$

Método da relaxação

iterativamente:

$$y_k^{(n+1)} = \frac{y_{k+1}^{(n)} + y_{k-1}^{(n)} - h^2 f(x_k, y_k^{(n)})}{2}$$

sobre-relaxação:

$$y_k^{(n+1)} = w \left(\frac{y_{k+1}^{(n)} + y_{k-1}^{(n)} - h^2 f(x_k, y_k^{(n)})}{2} \right) + (1 - w)y_k^{(n)}$$

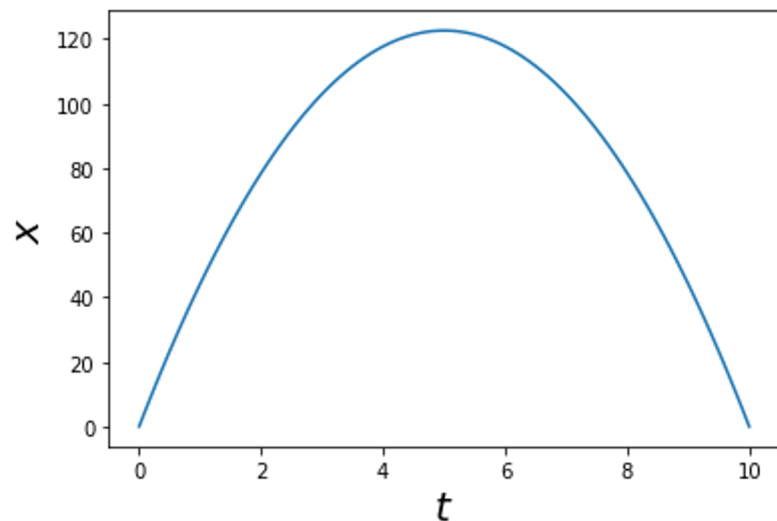
parâmetro de sobre-relaxação: $w \in [0, 2]$

Método da relaxação

$$\frac{d^2 x}{dt^2} = -g$$

$x = 0$ para $t = 0, 10$ s

```
1 from numpy import array, arange, ones, max
2 from matplotlib import pyplot as plt
3
4 g=9.81
5 t_i, t_f=0.0, 10.0
6 N=100 #number of interval in time
7 h=(t_f-t_i)/N
8
9 def f():
10     return (-g)
11
12 t=list(arange(t_i, t_f, h))
13 t.append(t[len(t)-1]+h)
14 t[0]=0.0
15 leng_t=len(t)
16 x=list(ones(leng_t, float))
17 i=1
18 for i in range(1, leng_t-1):
19     x[i]=20.0
20 x[0]=0.0
21 x[leng_t-1]=0.0
22 xtmp=copy(x)
23
24 tolerance=1e-6
25 delta=1.0
26 while delta>tolerance:
27     for i in range(leng_t):
28         if i==0 or i==leng_t-1:
29             xtmp[i]=x[i]
30         else:
31             xtmp[i]=(x[i+1]+x[i-1]-f()*h**2)/2.0
32     delta=max(abs(x-xtmp))
33     x, xtmp=xtmp, x
34
35 plt.plot(t, x)
36 plt.xlabel(r'$t$', fontsize=20)
37 plt.ylabel(r'$x$', fontsize=20)
38 plt.show()
```



- As equações diferenciais envolvendo derivadas parciais (PDEs) são de muito mais difícil resolução analítica que as ODEs
- E o mesmo se passa com a resolução numérica
- Podem-se usar algumas técnicas analíticas para transformar uma PDE num conjunto de ODEs que depois se resolve “normalmente”

2ª ordem, a duas dimensões

$$A(x, y) \frac{\partial^2 \Psi}{\partial x^2} + B(x, y) \frac{\partial^2 \Psi}{\partial x \partial y} + C(x, y) \frac{\partial^2 \Psi}{\partial y^2} + \\ + D(x, y) \frac{\partial \Psi}{\partial x} + E(x, y) \frac{\partial \Psi}{\partial y} + F(x, y) \Psi = G(x, y)$$

$$B^2 - 4AC < 0$$

PDE elíptica

Equação de Poisson

$$B^2 - 4AC = 0$$

PDE parabólica

Equação de difusão

$$B^2 - 4AC > 0$$

PDE hiperbólica

Equação de onda

- As condições fronteira das PDEs elípticas são tipicamente o valor da função na fronteira do domínio de integração

$$\frac{\partial^2 V}{\partial x^2} + \frac{\partial^2 V}{\partial y^2} = -\frac{\rho(x, y)}{\epsilon}$$

$$V(0, y) = 0$$

$$V(10, y) = 100$$

$$V(x, 0) = 0$$

$$V(x, 10) = 0$$

- As condições fronteira das PDEs hiperbólicas são tipicamente os valores iniciais da solução

$$\frac{\partial^2 \Psi}{\partial t^2} = c^2 \frac{\partial^2 \Psi}{\partial x^2}$$

$$\Psi(x, 0) = 10 \sin\left(\frac{\pi x}{10}\right)$$

$$\Psi'(x, 0) = \pi \cos\left(\frac{\pi x}{10}\right)$$

$$\Psi(-10, t) = -10$$

$$\Psi(10, t) = 10$$

Equações diferenciais parciais

Método da relaxação a duas dimensões:

$$\nabla^2 f = g(x, y)$$

$$\frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} = g(x, y)$$

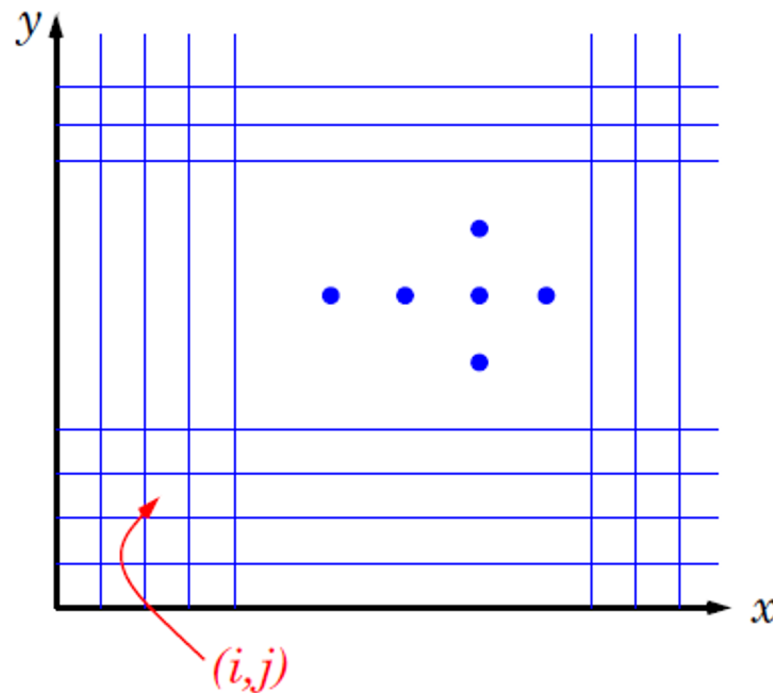
$$\frac{\partial^2 f}{\partial x^2} = \frac{f(x+h, y) + f(x-h, y) - 2f(x, y)}{h^2}$$

$$\frac{\partial^2 f}{\partial y^2} = \frac{f(x, y+h) + f(x, y-h) - 2f(x, y)}{h^2}$$

Equações diferenciais parciais

$$\frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} = \frac{f(x+h, y) + f(x-h, y) + f(x, y+h) + f(x, y-h) - 4f(x, y)}{h^2}$$

divisão da região 2D em pequenas áreas:



Equações diferenciais parciais

$$\frac{f(i+1, j) - 2f(i, j) + f(i-1, j)}{h^2} + \frac{f(i, j+1) - 2f(i, j) + f(i, j-1)}{h^2} = g(i, j)$$

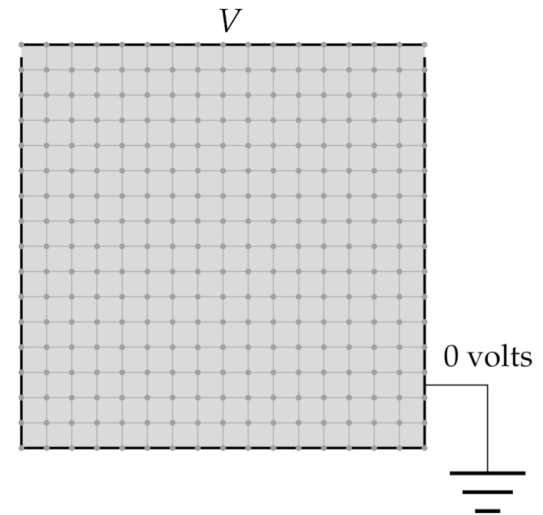
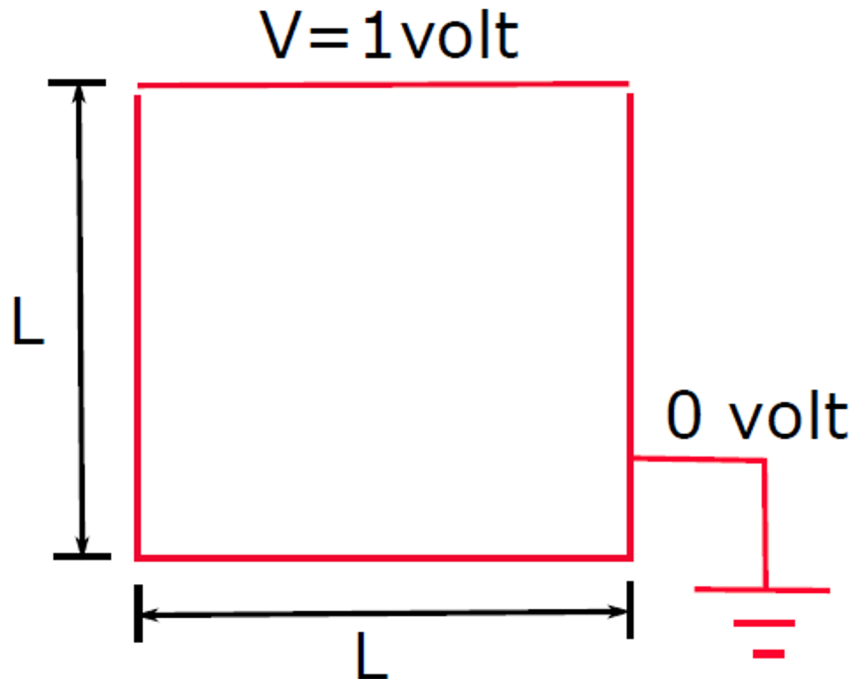
$$f^{(n+1)}(i, j) = \frac{f^{(n)}(i+1, j) + f^{(n)}(i-1, j) + f^{(n)}(i, j+1) + f^{(n)}(i, j-1) - h^2 g(i, j)}{4}$$

método da relaxação a 2D (Jacobi)

Equações diferenciais parciais

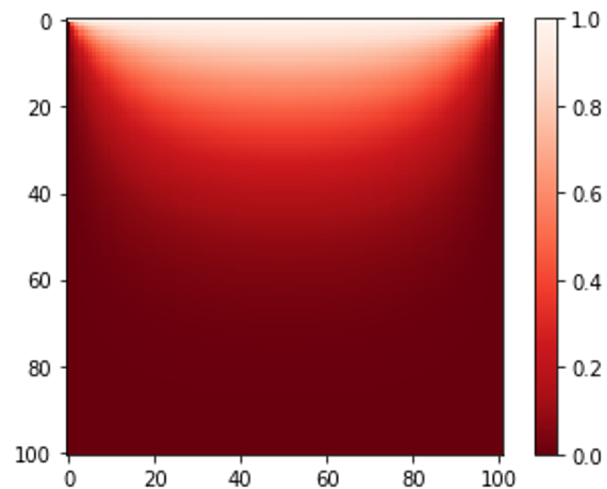
Exemplo: equação de Laplace

$$\nabla^2 \phi(x, y) = 0$$



Equações diferenciais parciais

```
1 from numpy import empty,zeros,max
2 from pylab import imshow,show,colorbar,set_cmap
3
4 # Constants
5 M = 100      # Grid squares on a side
6 V = 1.0      # Voltage at top wall
7 target = 1e-4 # Target accuracy
8
9 # Create arrays to hold potential values
10 phi = zeros([M+1,M+1],float)
11 phi[0,:] = V
12 phiprime = empty([M+1,M+1],float)
13
14 # Main loop
15 delta = 1.0
16 while delta > target:
17     # Calculate new values of the potential
18     for i in range(M+1):
19         for j in range(M+1):
20             if i==0 or i==M or j==0 or j==M:
21                 phiprime[i,j] = phi[i,j]
22             else:
23                 phiprime[i,j] = (phi[i+1,j] + phi[i-1,j] \
24                                 + phi[i,j+1] + phi[i,j-1])/4
25
26     # Calculate maximum difference from old values
27     delta = max(abs(phi-phiprime))
28
29     # Swap the two arrays around
30     phi,phiprime = phiprime,phi
31
32 # Make a plot
33 imshow(phi)
34 #print(phi)
35 colorbar()
36 set_cmap("Reds_r")
37 show()
```



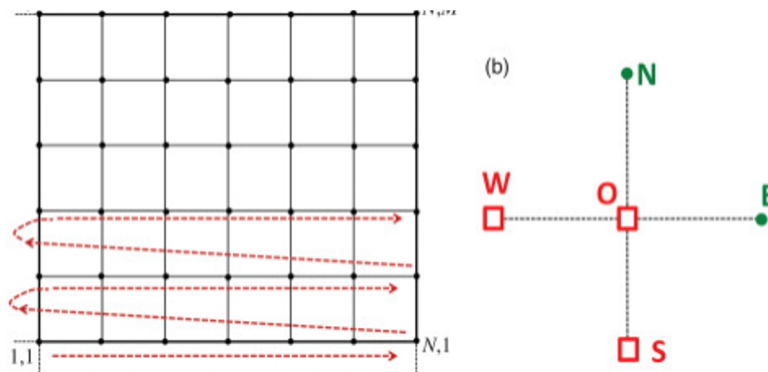
Sobre-relaxação a 2D:

$$f^{(n+1)}(i, j) = w \frac{f^{(n)}(i+1, j) + f^{(n)}(i-1, j) + f^{(n)}(i, j+1) + f^{(n)}(i, j-1) - h^2 g(i, j)}{4} + (1-w)f^{(n)}(i, j)$$

Método de Gauss-Seidel

O método de Jacobi para além de ser, em geral, instável (i.e. pode não convergir), é lento

Podemos melhorar a rapidez ao assumir que os novos valores são melhores que os anteriores, pelo que nas linhas seguintes podemos usar os últimos valores calculados



Método de Gauss-Seidel

$$\phi(x, y) \leftarrow \frac{1}{4}[\phi(x + a, y) + \phi(x - a, y) + \phi(x, y + a) + \phi(x, y - a)]$$

$$\phi(x, y) \leftarrow \frac{1 + \omega}{4}[\phi(x + a, y) + \phi(x - a, y) + \phi(x, y + a) + \phi(x, y - a)] - \omega\phi(x, y)$$

Equação da difusão

$$\frac{\partial \Phi}{\partial t} = D \frac{\partial^2 \Phi}{\partial x^2}$$

$$\Phi(x, t) \Rightarrow \Phi_{i,j}$$

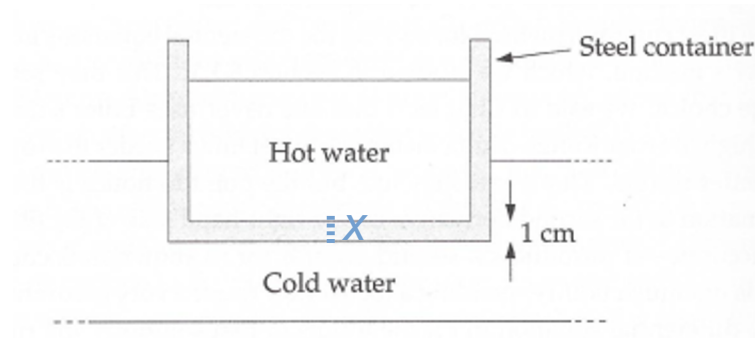
$$\frac{\Phi_{i,j+1} - \Phi_{i,j}}{dt} = D \frac{\Phi_{i+1,j} - 2\Phi_{i,j} + \Phi_{i-1,j}}{(dx)^2}$$

$$\Phi_{i,j+1} = D\alpha\Phi_{i+1,j} + (1 - 2D\alpha)\Phi_{i,j} + D\alpha\Phi_{i-1,j}$$

$$\alpha = \frac{dt}{(dx)^2}$$

Equações diferenciais parciais

Equação da difusão $\frac{\partial T}{\partial t} = D \frac{\partial^2 T}{\partial x^2}$



20°C

50°C

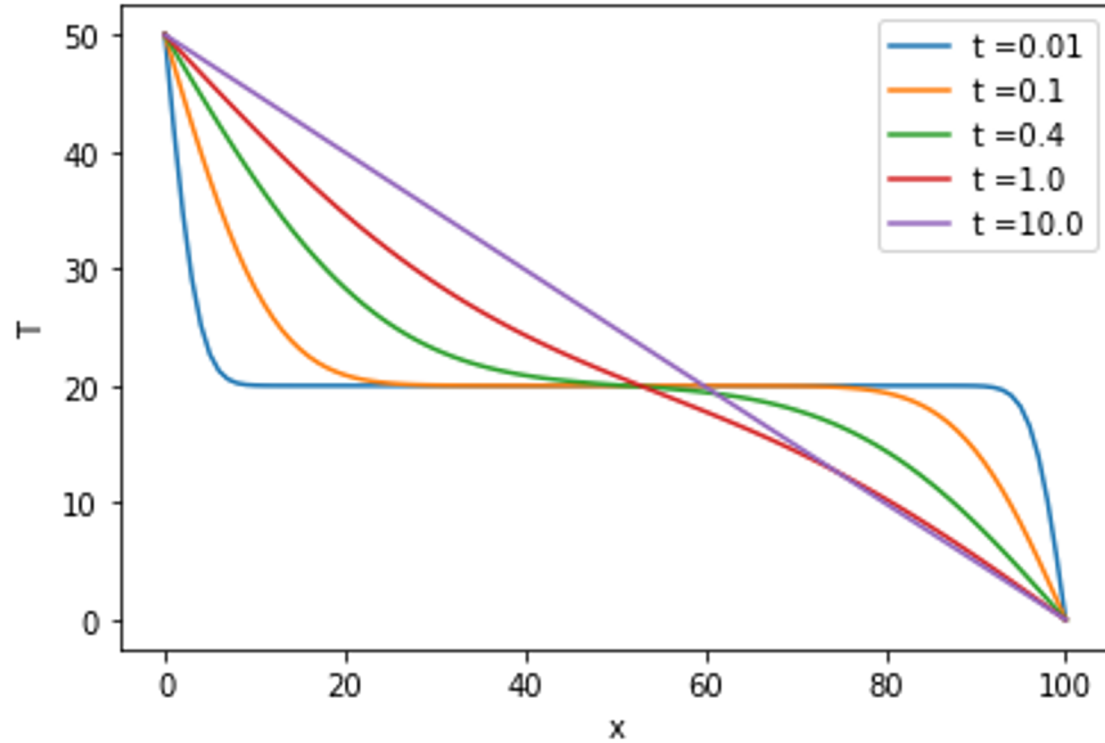
0°C

Equações diferenciais parciais

```
1 from numpy import empty
2 from pylab import plot,xlabel,ylabel,show,legend
3
4 # Constants
5 L = 0.01      # Thickness of steel in meters
6 D = 4.25e-6   # Thermal diffusivity
7 N = 100      # Number of divisions in grid
8 a = L/N      # Grid spacing
9 h = 1e-4     # Time-step
10 epsilon = h/1000
11
12 Tlo = 0.0     # Low temperature in Celcius
13 Tmid = 20.0   # Intermediate temperature in Celcius
14 Thi = 50.0   # Hi temperature in Celcius
15
16 t1 = 0.01
17 t2 = 0.1
18 t3 = 0.4
19 t4 = 1.0
20 t5 = 10.0
21 tend = t5 + epsilon
22
23 # Create arrays
24 T = empty(N+1,float)
25 T[0] = Thi
26 T[N] = Tlo
27 T[1:N] = Tmid
28 Tp = empty(N+1,float)
29 Tp[0] = Thi
30 Tp[N] = Tlo
31
```

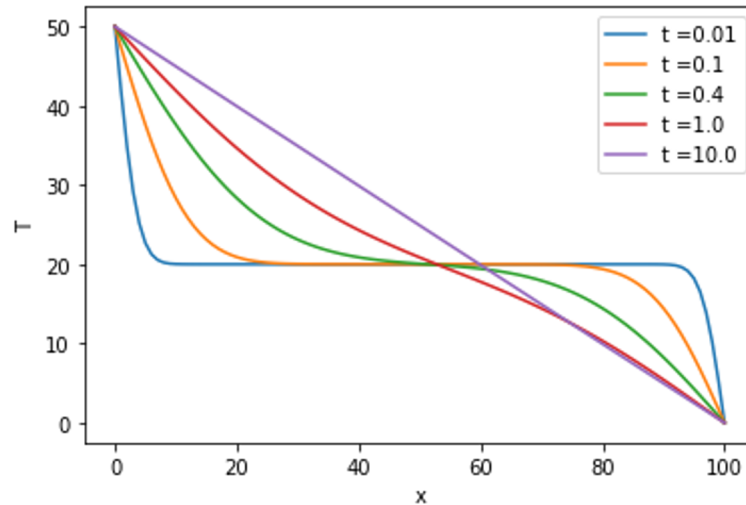
```
32 # Main loop
33 t = 0.0
34 c = h*D/(a*a)
35 while t<tend:
36
37     # Calculate the new values of T
38     for i in range(1,N):
39         Tp[i] = T[i] + c*(T[i+1]+T[i-1]-2*T[i])
40     T,Tp = Tp,T
41     t += h
42
43     # Make plots at the given times
44     if abs(t-t1)<epsilon:
45         plot(T,label='t='+str(t1))
46     if abs(t-t2)<epsilon:
47         plot(T,label='t='+str(t2))
48     if abs(t-t3)<epsilon:
49         plot(T,label='t='+str(t3))
50     if abs(t-t4)<epsilon:
51         plot(T,label='t='+str(t4))
52     if abs(t-t5)<epsilon:
53         plot(T,label='t='+str(t5))
54
55     xlabel("x")
56     ylabel("T")
57     legend()
58     show()
```

Equações diferenciais parciais



Equações diferenciais parciais

```
37 # Calculate the new values of T
38 # for i in range(1,N):
39 #     Tp[i] = T[i] + c*(T[i+1]+T[i-1]-2*T[i])
40 Tp[1:N] = T[1:N] + c*(T[2:N+1]+T[0:N-1]-2*T[1:N])
```



Equação de onda:
$$\frac{\partial^2 \phi}{\partial x^2} - \frac{1}{v^2} \frac{\partial^2 \phi}{\partial t^2} = 0 \quad \Leftrightarrow \quad \frac{\partial^2 \phi}{\partial t^2} = v^2 \frac{\partial^2 \phi}{\partial x^2}$$

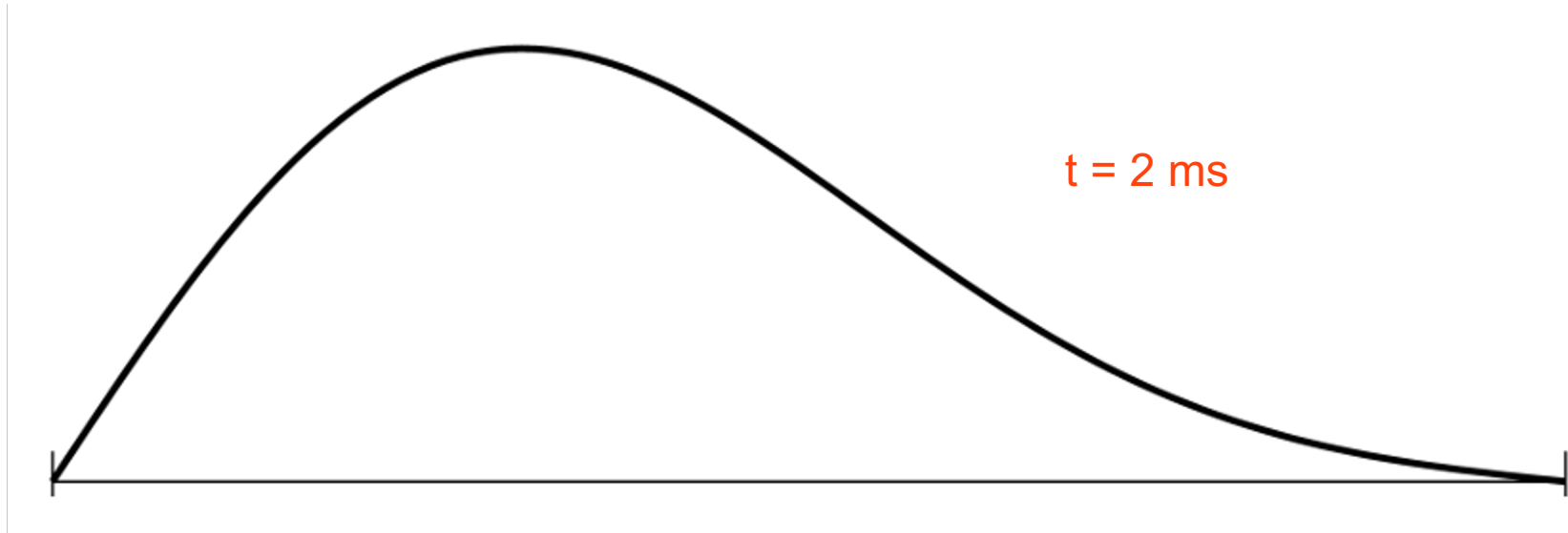
Usando a definição de 2ª derivada numérica:

$$\frac{\partial^2 \phi}{\partial t^2} = \frac{v^2}{a^2} [\phi(x + a, t) + \phi(x - a, t) - 2\phi(x, t)]$$

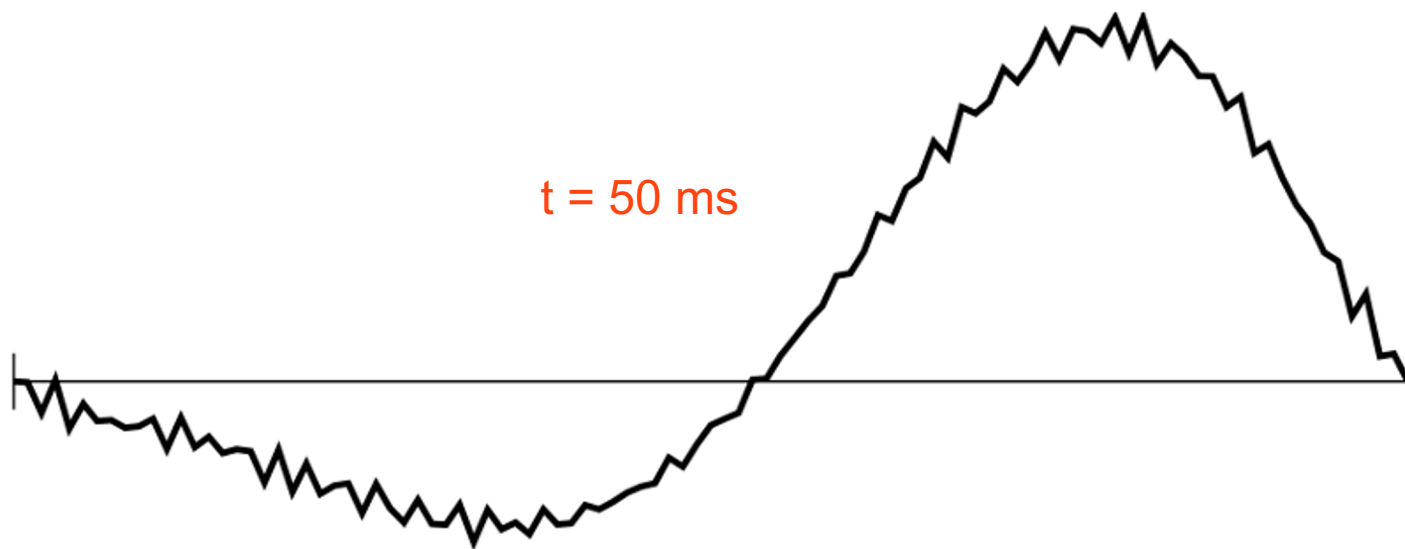
$$\frac{\partial \phi}{\partial t} = \psi(x, t), \quad \frac{\partial \psi}{\partial t} = \frac{v^2}{a^2} [\phi(x + a, t) + \phi(x - a, t) - 2\phi(x, t)]$$

Equações diferenciais parciais

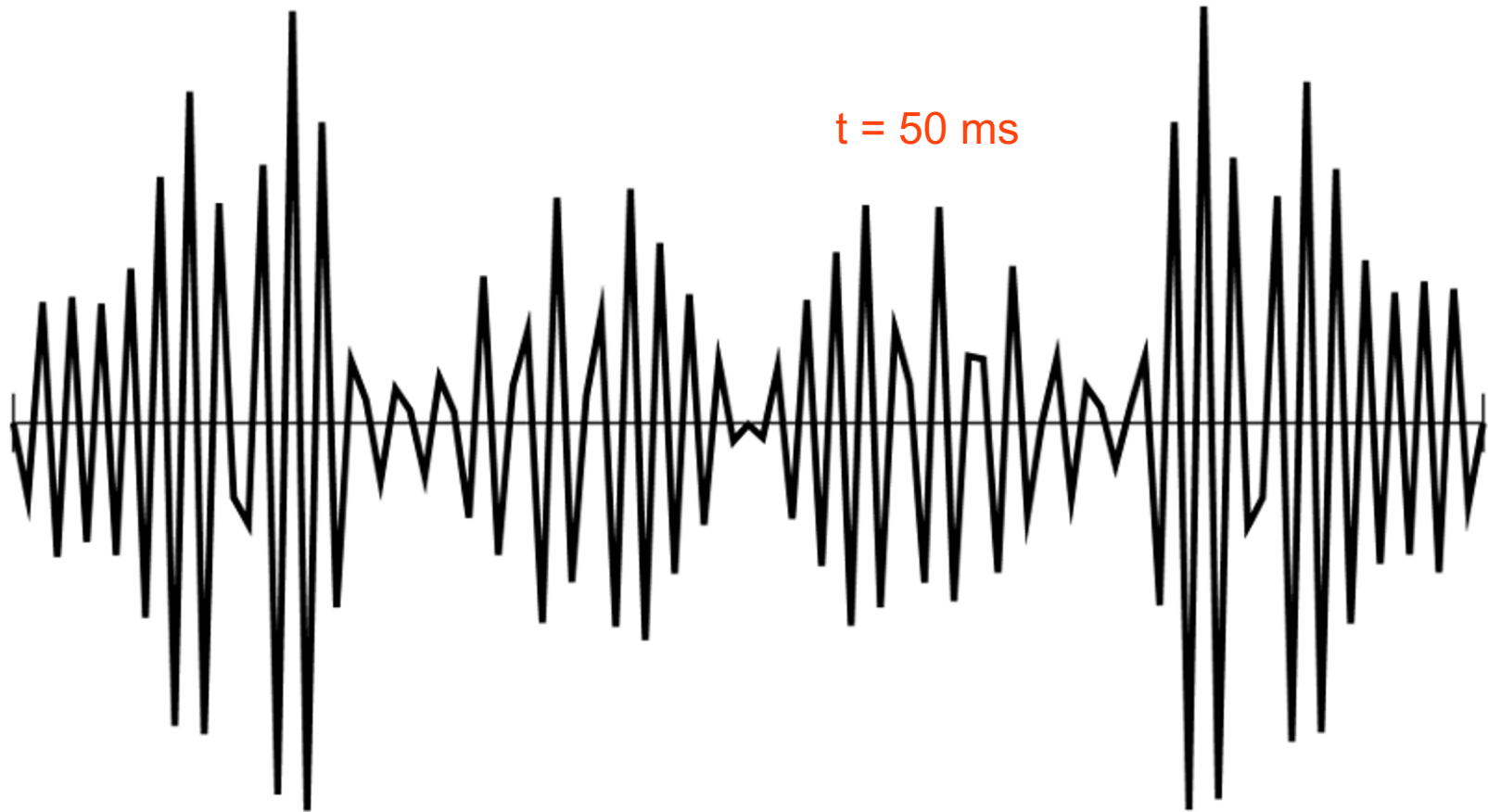
$$\begin{cases} \phi(x, t + h) = \phi(x, t) + h\phi(x, t) \\ \psi(x, t + h) = \psi(x, t) + h\frac{v^2}{a^2}[\phi(x + a, t) + \phi(x - a, t) - 2\phi(x, t)] \end{cases}$$



Equações diferenciais parciais



Equações diferenciais parciais



- Note-se que há dois passos de integração diferentes: dt e dx
- A estabilidade da propagação temporal depende da relação entre os passos, isto é, do coeficiente α
- Pode-se estudar a estabilidade começando com $\Phi(x,0)=\delta(x)$ e analisando a solução numérica (sabendo que a solução analítica, neste caso, é uma gaussiana cuja largura aumenta com a raiz quadrada de t)

- Conclui-se que a propagação só é estável se

$$dt < \frac{(dx)^2}{2D}$$